

Usecase

Usecase層の前提

項目	内容
Usecaseの役割	業務ルール、判断、計算、Repository呼び出し順の制御を担当する
Controllerの役割	Request DTOの受け取り、認証済みUserID/GuestSessionIDの取得、HTTPステータス変換を担当する
Usecaseが扱わないもの	Echo Context、HTTP status、Cookie、GORM、Redis Client
DB操作	Repository Interface経由で行う
Redis操作	Redis用Repository Interface経由で行う
Error	Usecaseは業務エラーを返し、ControllerがHTTP statusへ変換する
Event記録	フロントから直接送られる <code>content_view</code> 、 <code>impression</code> 、 <code>stay</code> 、 <code>click</code> 、 <code>re_search</code> は EventUsecase が受け取り、ActionEventRepository で記録する。これらはイベント記録自体が本体処理のため、ActionEvent作成に失敗した場合はエラーを返す。一方、保存・評価・モーダル操作の成功後に作成する <code>save</code> 、 <code>rating</code> 、 <code>modal_impression</code> 、 <code>modal_click</code> 、 <code>modal_close</code> は補助記録として扱う。 本体処理成功後にActionEvent作成を試みるが、ActionEvent作成に失敗しても本体処理はrollbackせず、APIは本体処理成功として返す。失敗内容はサーバーログに記録する。
Bean/Article詳細	詳細取得だけを行い、 <code>content_view</code> の直接記録はしない
Rating	最新Markdownに合わせ、Good = <code>+1</code> 、Bad = <code>-1</code> とする
Modal表示頻度	最終判断はBackendの <code>ModalUsecase</code> 。Frontendは表示タイミングの検知のみ

Usecase一覧

Usecase名	何をしているか	責務	使うRepository	説明
<code>AuthUsecase</code>	認証処理を行う	Signup、Login、Refresh、Logout、Me、token_version更新、reuse検知	<code>UserRepository</code> , <code>RefreshTokenRepository</code> , <code>AuditLogRepository</code>	ログインユーザーの認証、AccessToken再発行、RefreshToken rotationを担当する
<code>GuestSessionUsecase</code>	Guestを一時識別する	GuestSession作成、取得、延長	<code>GuestSessionRepository</code>	未ログインユーザーの行動ログを紐付けるための識別情報を管理する
<code>BeanUsecase</code>	Beanの表示・取得を行う	Bean一覧、Bean詳細、関連記事取得	<code>BeanRepository</code> , <code>BeanArticleRepository</code>	ユーザー向けBean表示処理を担当する。event記録はしない
<code>ArticleUsecase</code>	Articleの表示・取得を行う	Article一覧、Article詳細、関連Bean取得、Guest閲覧制御	<code>ArticleRepository</code> , <code>BeanArticleRepository</code>	Article一覧はGuest/User/Adminが閲覧可能。Article本文を含む詳細はUser/Adminのみ返す。Guestが詳細

Usecase名	何をしているか	責務	使うRepository	説明
				を開こうとした場合はログイン誘導するエラーを返す
SearchUsecase	Bean / Articleを検索する	検索条件確認、検索条件正規化、公開コンテンツ検索	BeanRepository , ArticleRepository	条件に合う公開コンテンツを返す。re_search記録は行わない。検索条件変更イベントはPOST /eventsでEventUsecaseが記録する
EventUsecase	行動イベントを記録する	content_view、impression、stay、click、re_search	ActionEventRepository 、 RankTargetRepository 、 EventDedupRepository	ランキング計算・興味スコア計算の元になる通常行動ログを保存する。save / rating / modal系eventは直接受け付けず、それぞれ専用Usecaseで作成する
SavedItemUsecase	保存・保存解除を行う	Save、Remove、保存一覧、保存済み確認	SavedItemRepository 、 RankTargetRepository 、 ActionEventRepository	User/AdminがBean/Articleの保存・保存解除を行う処理を担当。保存本体はSavedItemRepositoryで処理する。保存成功後にActionEventRepositoryでsave eventの作成を試みるが、save event作成に失敗しても保存本体はrollbackせず、APIは保存成功として返す。event作成失敗時はサーバーログに記録する。保存解除はSavedItemRepositoryの単体処理で完結し、save eventは作成しない。
RatingUsecase	Good / Bad評価を行う	評価、再評価、評価取得	RatingRepository 、 RankTargetRepository 、 ActionEventRepository	User/AdminがBean/ArticleにGood/Bad評価を付ける処理を担当。評価本体はRatingRepository.Upsertで処理する。評価成功後にActionEventRepositoryでrating eventの作成を試みるが、rating event作成に失敗しても評価本体はrollbackせず、APIは評価成功として返す。失敗内容はサーバーログに記録する。
RecommendationUsecase	推薦一覧を作る	推薦一覧取得、推薦理由作成、スコア順整列、検索条件正規化、filter正規化	ContentMetricRepository 、 InterestProfileRepository 、 RankTargetRepository 、 SavedItemRepository 、 BeanRepository 、 ArticleRepository 、 ActionEventRepository	ユーザーまたはGuestの興味と計算済みmetricsをもとに推薦一覧を作る
RankingUsecase	ランキングを取得する	Beanランキング、Articleランキング、TOPランキング取得	ContentMetricRepository 、 RankTargetRepository 、 BeanRepository 、 ArticleRepository	計算済みmetricsをもとに表示用ランキングを作る
ModalUsecase	推薦モーダルを制御する	表示可否判定、候補取得、block_reason、クリック、閉じる	ModalDisplayLogRepository 、 ContentMetricRepository 、 InterestProfileRepository 、 SavedItemRepository 、 ActionEventRepository 、 ModalSuppressionRepository	推薦モーダルの表示可否判定、候補取得、表示ログ作成、クリック、閉じる処理。表示ログ作成、クリック更新、閉じる更新はModalDisplayLogRepositoryで処理する。クリック/閉じる更新は、actor条件付きUPDATEにより、本人または本人GuestSessionのログだけを更新する。modal eventの作成は本体処理成功後に試みるが、modal event作成に失敗しても本体処理はrollbackせず、APIは成功として返す。失敗内容はサーバーログに記録する。

Usecase名	何をしているか	責務	使うRepository	説明
RankingBatchUsecase	ランキング指標を再計算する	content_metrics更新、ランキングscore計算、BatchRun更新	ActionEventRepository , ContentMetricRepository , BatchRunRepository , BatchLockRepository , AuditLogRepository	行動ログを集計し、ランキング表示に使うContentMetricを更新する処理。ContentMetric更新とBatchRun成功更新は、集計結果とバッチ成功状態を一致させるためTxで一体化。AuditLog作成はTxに含めず、Batch本体成功後に作成を試みる。AuditLog作成に失敗してもBatch本体はrollbackせず、失敗内容はサーバーログに記録する。
AdminBeanUsecase	管理者がBeanを管理する	Bean作成、更新、公開、非公開	BeanRepository , RankTargetRepository , AuditLogRepository	Admin画面からBeanの作成、更新、公開、非公開を行う処理を担当する。作成・更新はBeanRepositoryの単体処理で行い、Txは使わない。公開/非公開時は、Beanの公開状態とRankTargetの有効状態がズレると表示制御が壊れるため、Bean.UpdatePublishedとRankTarget更新をTxで一体化する。AuditLog作成はTxに含めず、本体処理成功後に作成を試みる
AdminArticleUsecase	管理者がArticleを管理する	Article作成、更新、公開、非公開	ArticleRepository , RankTargetRepository , AuditLogRepository	Admin画面からArticleの作成、更新、公開、非公開を行う処理を。作成・更新はArticleRepositoryの単体処理で行い、Txは使わない。公開/非公開時は、Articleの公開状態とRankTargetの有効状態がズレると表示制御が壊れるため、Article.UpdatePublishedとRankTarget更新をTxで一体化する。AuditLog作成はTxに含めず、本体処理成功後に作成を試みる
AdminRelationUsecase	BeanとArticleの関連を管理する	関連付け作成、削除、表示順更新	BeanRepository , ArticleRepository , BeanArticleRepository , AuditLogRepository	BeanとArticleの関連を管理する処理を担当する。関連1件の作成・削除はBeanArticleRepositoryの単体処理で行い、Txは使わない。一括差し替え・並び替えは、既存関連の削除と新規関連の作成をまとめて行うため、途中失敗で関連状態が壊れないようにTxで処理する。AuditLog作成はTxに含めず、本体処理成功後に作成を試みる。
AdminBatchUsecase	管理者がバッチを操作する	手動バッチ実行、実行履歴取得	BatchRunRepository , BatchLockRepository , AuditLogRepository , RankingBatchUsecase , InterestBatchUsecase	Admin画面からランキング再計算・興味プロフィール再計算を手動実行する処理を担当する。実際の集計処理はRankingBatchUsecaseとInterestBatchUsecaseへ委譲する。BatchLockで二重実行を防ぎ、BatchRunで実行履歴を記録する。AuditLog作成は本体処理成功後に試みるが、AuditLog作成に失敗してもバッチ本体はrollbackしない。
InterestBatchUsecase	興味スコアを再計算する	interest_profiles更新、User/Guest別の興味score計算	ActionEventRepository , InterestProfileRepository , BatchRunRepository , BatchLockRepository , AuditLogRepository	行動ログをもとにUser/Guestごとの興味プロフィールを再計算する処理を担当。InterestProfileは再計算可能な派生データのため、BulkUpsertはTx不要とする。途中で失敗した場合はエラーとして記録し、次回バッチで再計算できるようにする。
AdminAuditUsecase	監査ログを確認する	AuditLog一覧、詳細、検索	AuditLogRepository	管理者が操作履歴やバッチ履歴を確認する

Usecase名	何をしているか	責務	使うRepository	説明
<code>CleanupUsecase</code>	不要データを掃除する	期限切れ RefreshToken、 GuestSession、 Guest InterestProfile削除	<code>RefreshTokenRepository</code> , <code>GuestSessionRepository</code> , <code>InterestProfileRepository</code>	
<code>RateLimitUsecase</code>	API利用回数を制限する	key生成補助、 limitルール選択、 TokenBucket判定	<code>RateLimitRepository</code>	Middlewareから呼ばれ、対象APIごとのRateLimit可否を判定する。 Redis Clientは直接扱わない
<code>AdminRateLimitUsecase</code>	管理者がRateLimit状態を操作する	RateLimit状態確認、reset	<code>RateLimitRepository</code> , <code>AuditLogRepository</code>	Adminが特定keyのRateLimit状態をリセットする場合に使う

transaction必要なUsecase一覧

Usecase	transaction必要	Tx内でまとめる処理	Txに入れない処理
<code>AuthUsecase.Signup</code>	不要	なし	User.ExistsByEmail
<code>AuthUsecase.Login</code>	不要	なし	User.FindByEmail RefreshToken.Create AuditLog.Create
<code>AuthUsecase.Refresh</code>	必須	RefreshToken.FindByTokenHashWithUserForUpdate、 RefreshToken.Create、RefreshToken.MarkUsed	Cookie設定、AuditLog.Create
<code>AuthUsecase.RefreshReuseDetected</code>	必須	RefreshToken.RevokeFamily、 User.IncrementTokenVersion	Cookie削除、AuditLog.Create
<code>AuthUsecase.LogoutCurrentFamily</code>	不要	なし	RefreshToken.FamilyDelete RefreshToken.FamilyDelete AuditLog.Create
<code>AuthUsecase.LogoutAllDevices</code>	必須	RefreshToken.RevokeByUserID、 User.IncrementTokenVersion	Cookie削除、AuditLog.Create
<code>SavedItemUsecase.Save</code>	不要	なし	RankTarget.Exists SavedItem.Save ActionEvent.Create

Usecase	transaction必要	Tx内でまとめる処理	Txに入れない処
<code>SavedItemUsecase.Remove</code>	不要	なし	<code>SavedItem.Remove</code>
<code>RatingUsecase.Rate</code>	不要	なし	<code>RankTarget.Exist</code> <code>Rating.Upsert</code> 、 <code>ActionEvent.Create</code>
<code>RatingUsecase.DeleteRating</code>	不要	なし	<code>Rating.Delete</code>
<code>EventUsecase.CreateContentView</code>	不要	なし	<code>RankTarget.Exist</code> <code>EventDedup.Create</code> <code>ActionEvent.Create</code>
<code>EventUsecase.CreateImpression</code>	不要	なし	<code>RankTarget.Exist</code> <code>EventDedup.Create</code> <code>ActionEvent.Create</code>
<code>EventUsecase.CreateStay</code>	不要	なし	<code>RankTarget.Exist</code> <code>ActionEvent.Create</code>
<code>EventUsecase.CreateClick</code>	不要	なし	<code>RankTarget.Exist</code> <code>ActionEvent.Create</code>
<code>EventUsecase.CreateReSearch</code>	不要	なし	<code>ActionEvent.Create</code>
<code>ModalUsecase.ShowModal</code>	不要	なし	<code>ModalDisplayLogic</code> <code>ActionEvent.Create</code> (best effort)、 <code>ModalSuppress</code>
<code>ModalUsecase.ClickModal</code>	不要	なし	<code>ModalDisplayLogic</code> <code>ActionEvent.Create</code> (best effort)
<code>ModalUsecase.CloseModal</code>	不要	なし	<code>ModalDisplayLogic</code> <code>ActionEvent.Create</code> (best effort)、 <code>ModalSuppress</code>
<code>AdminBeanUsecase.CreateBean</code>	不要	なし	<code>Bean.Create</code> 、(best effort)
<code>AdminBeanUsecase.UpdateBean</code>	不要	なし	<code>Bean.Update</code> 、 (best effort)
<code>AdminBeanUsecase.PublishBean</code>	必須	<code>Bean.UpdatePublished</code> 、 <code>RankTarget.FindOrCreate</code> 、 <code>RankTarget.UpdateActive</code>	<code>AuditLog.Create</code>
<code>AdminBeanUsecase.UnpublishBean</code>	必須	<code>Bean.UpdatePublished</code> 、 <code>RankTarget.UpdateActive</code>	<code>AuditLog.Create</code>

Usecase	transaction必要	Tx内でまとめる処理	Txに入れない処
<code>AdminArticleUsecase.CreateArticle</code>	不要	なし	Article.Create、 effort)
<code>AdminArticleUsecase.UpdateArticle</code>	不要	なし	Article.Update、 effort)
<code>AdminArticleUsecase.PublishArticle</code>	必須	Article.UpdatePublished、RankTarget.FindOrCreate、 RankTarget.UpdateActive	AuditLog.Creat
<code>AdminArticleUsecase.UnpublishArticle</code>	必須	Article.UpdatePublished、RankTarget.UpdateActive	AuditLog.Creat
<code>AdminRelationUsecase.CreateRelation</code>	不要	なし	Bean.ExistsByI BeanArticle.Cre AuditLog.Creat
<code>AdminRelationUsecase.DeleteRelation</code>	不要	なし	BeanArticle.Del AuditLog.Creat
<code>AdminRelationUsecase.ReplaceRelations</code>	必須	BeanArticle.ReplaceByBeanID	Bean.ExistsByI AuditLog.Creat
<code>AdminRelationUsecase.UpdateDisplayOrder</code>	必須	BeanArticle.ReplaceByBeanID	Bean.ExistsByI AuditLog.Creat
<code>RankingBatchUsecase.Recalculate</code>	必須部分あり	ContentMetric.BulkUpsert、BatchRun.MarkSuccess	BatchLock.Acq BatchLock.Rele BatchRun.Mark AuditLog.Creat
<code>InterestBatchUsecase.Recalculate</code>	不要	なし	ActionEvent.Ag ActionEvent.Ag InterestProfile.f BatchRun.Mark BatchRun.Mark AuditLog.Creat
<code>CleanupUsecase.DeleteExpired</code>	不要	なし	RefreshToken.I GuestSession.I InterestProfile.I
<code>AdminRateLimitUsecase.Reset</code>	不要	なし	RateLimit.Reset effort)

transaction必須対応

処理	必須対応	理由
RefreshToken取得	Refresh rotationではtransaction内で取得する	rotation中の競合を防ぐため
RefreshToken行lock	<code>SELECT ... FOR UPDATE</code> 相当でlockする	同じRefreshTokenの二重使用を防ぐため

処理	必須対応	理由
MarkUsed条件	<code>WHERE id = ? AND used_at IS NULL AND revoked_at IS NULL</code>	使用済み・失効済みtokenを再更新しないため
MarkUsed更新件数	0件なら失敗、またはreuse疑いとして扱う	競合や再利用を見逃さないため
新RefreshToken作成	旧RefreshTokenのMarkUsedと同じtransaction内で作成する	新旧token状態のズレを防ぐため
Refresh正常rotation	<code>FindByTokenHashWithUserForUpdate → Create → MarkUsed</code> を同じtransactionで行う	旧token使用済み化と新token作成の片方だけ成功する状態を防ぐため
Refresh reuse検知	<code>RevokeFamily → IncrementTokenVersion</code> を同じtransactionで行う	RefreshToken family失効と既存AccessToken失効を一致させるため
LogoutCurrentFamily	transaction不要。 <code>RevokeFamily</code> 単体で行う	現在端末のRefreshToken family失効だけで完結するため
LogoutAllDevices	<code>RevokeByUserID → IncrementTokenVersion</code> を同じtransactionで行う	全RefreshToken失効と既存AccessToken失効を一致させるため
Save	transaction不要。 <code>SavedItem.SaveOrRestore</code> 成功後に <code>ActionEvent.Create(save)</code> を試みる	save event失敗で保存本体をrollbackしないため
Rating	transaction不要。 <code>Rating.Upsert</code> 成功後に <code>ActionEvent.Create(rating)</code> を試みる	rating event失敗で評価本体をrollbackしないため
Modal表示	transaction不要。 <code>ModalDisplayLog.Create</code> 成功後に <code>ActionEvent.Create(modal_impression)</code> を試みる	modal event失敗で表示ログをrollbackしないため
Modalクリック	transaction不要。 <code>MarkClickedForActor</code> の条件付きUPDATEで本人確認と更新を同時に行う	IDOR対策はTxではなくactor条件付きUPDATEで守るため
Modal閉じる	transaction不要。 <code>MarkClosedForActor</code> の条件付きUPDATEで本人確認と更新を同時に行う	IDOR対策はTxではなくactor条件付きUPDATEで守るため
Redis抑制保存	DB transactionに含めない	RedisはDB transactionでrollbackできないため
Batch成功更新	<code>ContentMetric.BulkUpsert → BatchRun.MarkSuccess</code> だけを同じtransactionで行う	集計結果と成功ステータスを一致させるため
Batch失敗更新	transaction外で <code>BatchRun.MarkFailed</code> を行う	失敗記録までrollbackされるのを防ぐため
AuditLog作成	原則transaction外で、本体処理成功後に作成を試みる	AuditLog失敗で本体処理をrollbackしないため
Cookie設定・削除	Controllerで行う	HTTPレスポンス処理でありDB transactionではないため

LogoutCurrentFamily 処理手順

順番	処理	Repository	rollback条件	エラー対応
1	入力受け取り	なし	Txなし	tokenがない場合でもCookie削除して204でよい
2	token hash化	なし	Txなし	hash化失敗ならErrInvalidToken
3	RefreshToken取得	RefreshTokenRepository.FindByTokenHashWithUser	Txなし	見つからない場合でもCookie削除して204でよい
4	所有者確認	なし	Txなし	user_id不一致ならErrUnauthorized
5	状態確認	なし	Txなし	期限切れ・失効済み・使用済みでもCookie削除して204でよい
6	family失効	RefreshTokenRepository.RevokeFamily	Txなし	DB更新失敗ならErrRevokeFailed
7	監査ログ作成	AuditLogRepository.Create	Txなし	失敗してもlogout本体は成功。サーバーログに記録

順番	処理	Repository	rollback条件	エラー対応
8	Cookie削除	なし	Txなし	ControllerでRefreshToken Cookie / CSRF Cookieを削除
9	結果返却	なし	なし	204 No Content

注意：

通常logoutではusers.token_versionを更新しない。現在端末のRefreshToken familyのみを失効する。

既存AccessTokenを強制失効する必要がある場合は、LogoutAllDevicesを使う。

LogoutAllDevices 処理手順

順番	処理	Repository	transaction	エラー対応
1	認証済みUser確認	なし	Tx開始前	user_idなしならErrUnauthorized
2	transaction開始	TxManager.WithinTx	開始	Tx開始失敗ならErrTransactionFailed
3	user全RefreshToken失効	RefreshTokenRepository.RevokeByUserID	Tx内	DB更新失敗ならrollbackしてErrRevokeFailed
4	token_version増加	UserRepository.IncrementTokenVersion	Tx内	DB更新失敗ならrollbackしてErrUpdateFailed
5	commit	TxManager	Tx終了	commit失敗ならErrTransactionFailed
6	監査ログ作成	AuditLogRepository.Create	Tx外	失敗してもlogout本体は成功。サーバーログに記録
7	Cookie削除	なし	Tx外	ControllerでCookie削除
8	結果返却	なし	なし	204 No Content

Usecaseごとの処理手順AuthUsecase

- AuthUsecaseでは、RefreshToken rotation、RefreshToken reuse検知、logout、全端末logoutにおいて、RefreshTokenの状態と `users.token_version` の扱いを明確に分ける。
- 通常のRefresh成功時は、正常なtoken rotationであるため、RefreshToken family全失効も `users.token_version` 増加も行わない。
 - この場合は、新RefreshTokenを作成し、旧RefreshTokenに `used_at` と `replaced_by_token_id` を設定する。
- RefreshToken reuse検知時は、使用済みRefreshTokenが再利用された状態であり、盗難疑いとして扱う。
 - この場合は、同じfamily_idのRefreshTokenを全失効し、`users.token_version` を増加させる。
 - `users.token_version` を増加させる理由は、既存AccessTokenも強制失効させるためである。
- 通常logoutでは、現在使っているRefreshToken familyのみを失効する。
 - `users.token_version` を更新しない。
 - 理由は、通常logoutは現在端末のRefreshTokenを止める処理であり、他端末のAccessTokenまで止める必要がないため。
- 全端末logoutでは、対象userのRefreshTokenをすべて失効し、`users.token_version` を増加させる。
 - 理由は、全端末のRefreshTokenと既存AccessTokenをまとめて止める必要があるため。

処理	RefreshToken family失効	user全RefreshToken失効	token_version増加	理由
Signup成功	しない	しない	しない	Signupではログイン状態にしないため
Login成功	しない	しない	しない	新しいRefreshToken familyを発行するだけのため
Refresh成功	しない	しない	しない	正常なtoken rotationのため

処理	RefreshToken family失効	user全 RefreshToken失効	token_version増加	理由
Refresh reuse検知	する	しない	する	すでにused_atが入ったrefresh tokenが再送されたつまり盗難疑いとして既存AccessTokenも止めるため
LogoutCurrentFamily	する	しない	しない	現在端末のRefreshToken familyだけ止めればよい
LogoutAllDevices	する	する	する	全端末のRefreshTokenとAccessTokenを止めるため
User停止	する	する	する	suspended / deleted userの既存AccessTokenも止めるため

Signup

Signup成功時に AccessToken / RefreshToken は発行しない。

Cookieを設定しない

RefreshTokenRepository.Create を呼ばない

順番	処理	内容	Repository
1	入力受け取り	ControllerからSignup入力値を受け取る	なし
2	email正規化	emailを小文字化し、前後の空白を取り除く	なし
3	email重複確認	同じemailのユーザーが存在しないか確認する	UserRepository.ExistsByEmail
4	password hash化	生Passwordを保存せず、hash化する	なし
5	User作成	usersに新規ユーザーを作成する	UserRepository.Create
6	結果返却	作成されたUserIDを返す	なし

Login

順番	処理	何をしているか	Repository呼び出し	説明	主なエラー
1	emailでUser取得	ログイン対象のUserを探す	<code>UserRepository.FindByEmail</code>	emailは小文字正規化して検索	<code>ErrInvalidCredentials</code>
2	Status確認	ログイン可能なUserか確認する	なし	<code>active</code> 以外はログイン不可	<code>ErrUserSuspended</code> , <code>ErrUserDeleted</code>
3	password照合	入力passwordとhashを比較する	なし	不一致でも詳細理由は返さない	<code>ErrInvalidCredentials</code>
4	RefreshToken作成	新しいRefreshTokenを作成する	<code>RefreshTokenRepository.Create</code>	family_idを発行する	<code>ErrCreateFailed</code>
5	監査ログ作成	login成功を記録する	<code>AuditLogRepository.Create</code>	認証ログとして記録する。ただしAuditLog作成に失敗してもLogin本体はrollbackせず、APIはLogin成功として返す。失敗内容はサーバーログに記録する	なし
6	結果返却	AccessToken生成情報とRefreshToken生成値を返す	なし	JWT生成やCookie設定の境界は設計で分ける	なし

Refresh

Refreshは、RefreshToken rotationを行い、新しいAccessTokenと新しいRefreshTokenを発行する。

古いRefreshTokenは used_at を設定し、ReplacedByTokenID に新RefreshTokenのIDを保存する。

reuse検知時は同じfamily_idのRefreshTokenを全失効し、token_versionを更新して既存AccessTokenも無効化する。

AccessTokenの期限切れは、RefreshTokenが有効なら再発行できる。

RefreshToken自体が期限切れなら、AccessTokenは再発行できない。

その場合は、サーバーは再ログイン要求を返し、クライアント側ではCookieや認証状態を削除してログアウト状態にする

順番	処理	何をしているか	Repository呼び出し	説明
1	RefreshToken hash化	Cookieから受け取った生RefreshTokenをhash化する	なし	DBには生tokenを保存しない
2	transaction開始	rotation中の部分成功を防ぐ	TxManager.WithinTx	新token作成、旧token更新につまとめ
3	token取得 + lock	token_hashでRefreshTokenを取得し、行をlockする	RefreshTokenRepository.FindByTokenHashWithUserForUpdate	二重refreshを防ぐ
4	User確認	token所有Userが有効か確認する	なし	User status, token_versionに使う
5	期限確認	expires_at が now より後か確認する	なし	期限切れは行不可
6	失効確認	revoked_at が nil か確認する	なし	失効済みtokenは使用不可
7	reuse検知	used_at が nil か確認する	なし	used_atあり、再利用とみ
8	reuse時family失効	使用済みtoken再利用なら同familyを失効	RefreshTokenRepository.RevokeFamily	不正利用疑、RefreshToken familyをまとめて止める
9	reuse時 token_version更新	既存AccessTokenも無効化する	UserRepository.IncrementTokenVersion	盗まれたtokenのあるAccessTokenを止める
10	reuse時監査ログ作成	refresh reuse検知を記録する	AuditLogRepository.Create	セキュリティ査用に記録する。失敗し、reuse対応はrollbackしない。失敗内サーバーログ記録する
11	新RefreshToken作成	同じfamily_idで新tokenを作る	RefreshTokenRepository.Create	token_hashをDB保存
12	旧RefreshToken使用済み化	古いtokenにused_atとreplaced_by_token_idを入れる	RefreshTokenRepository.MarkUsed	rotation履歴残す
13	結果返却	再ログイン要求を返す	なし	新RefreshTokenは作成しない

reuse検知時は、手順8～10を実行した後、新tokenは作成せず401を返す。

正常rotation時は、手順11～12を同一transaction内で行う。

TX

順番	処理	何をしているのか	なぜ必要か	状態確認	Repository
1	生RefreshToken取得	Cookieから送られたRefreshTokenを受け取る	tokenがないと本人確認できないため	tokenが空でない	なし

順番	処理	何をしているのか	なぜ必要か	状態確認	Repository
2	token hash化	生TokenをDB保存形式のhashへ変換する	DBには生Tokenを保存しないため	hash化できる	なし
3	transaction開始	Refresh処理を1つのDB処理として扱う	旧Token更新と新Token作成を一体化するため	Tx開始成功	TxManager.WithinTx
4	RefreshToken取得 + lock	token_hashに一致するRefreshTokenを行ロック付きで取得する	同じTokenが同時に使われる競合を防ぐため	token_hashが存在する	RefreshToken.FindByTokenHashWithUserFor
5	User状態確認	tokenに紐づくUserが利用可能か確認する	suspended / deleted userにtokenを再発行しないため	User.status = active	なし
6	期限確認	RefreshTokenの期限が切れていないか確認する	期限切れTokenを使わせないため	expires_at > now	なし
7	失効確認	revoked_atが入っていないか確認する	logout済み・強制失効済みtokenを使わせないため	revoked_at == nil	なし
8	使用済み確認	used_atが入っていないか確認する	rotation済みTokenの再利用を検知するため	used_at == nil	なし
9	新RefreshToken作成	同じfamily_idで新しいRefreshTokenを作成する	RefreshToken rotationを成立させるため	user_id / family_id が有効	RefreshToken.Create
10	旧RefreshToken使用済み化	旧Tokenにused_atとreplaced_byを入れる	同じRefreshTokenを再利用できないようにするため	used_atがまだnil	RefreshToken.MarkUsed
11	監査ログ作成	refresh成功を記録する	認証操作の追跡に必要なため	user_idあり	AuditLog.Create
12	commit	DB変更を確定する	全処理が成功した場合のみ反映するため	全処理成功	TxManager
13	Cookie設定	新RefreshTokenをCookieへ設定する	次回refreshで新Tokenを使わせるため	commit済み	なし

Logout

Logoutは、現在使っているRefreshToken familyを失効し、Cookie削除用の成功結果を返す。

全端末logoutとは分ける。

通常logoutでは RevokeFamily(family_id) を使う。

全端末logoutでは RevokeByUserID(user_id) を使う。

順番	処理	何をしているか	Repository呼び出し	説明	主なエラー
1	User確認	認証middlewareからUserIDを受け取る	なし	Bodyのuser_idは信用しない	ErrUnaut
2	RefreshToken hash化	Cookieのrefresh_tokenをhash化する	なし	現在のfamilyを特定するため	なし

順番	処理	何をしているか	Repository呼び出し	説明	主なエラー
3	token取得	token_hashでRefreshTokenを取得する	RefreshTokenRepository.FindByTokenHashWithUser	tokenがなくてもlogout成功扱いにできる	なし
4	所有者確認	token.UserID と認証済みUserIDが一致するか確認する	なし	他人のtokenでlogoutしない	ErrUnaut
5	transaction開始	family失効と監査ログをまとめる	TxManager.WithinTx	部分成功を防ぐ	ErrTrans:
6	RefreshToken family失効	現在のfamilyをまとめて失効する	RefreshTokenRepository.RevokeFamily	現在端末のrefreshを止める	ErrRevok
7	監査ログ作成	logoutを記録する	AuditLogRepository.Create	認証ログを残す	ErrAuditl
8	commit	DB変更を確定する	TxManager	失効と監査ログが両方成功した場合のみ確定	ErrTrans:
9	Cookie削除	refresh_token / csrf cookieを削除する	なし	Controllerで行う	なし
10	結果返却	204を返す	なし	Cookieがなくても成功扱いでよい	なし

Cookieにrefresh_tokenがない場合、またはDBにtokenがない場合でも、ControllerはCookie削除を行い204を返してよい。

TX

順番	処理	何をしているのか	なぜ必要か	状態確認	Repository	re
1	認証済みUser確認	AccessTokenからuser_idを取得する	誰のlogoutか判断するため	user_idあり	なし	T re
2	生RefreshToken取得	CookieからRefreshTokenを取得する	失効対象のfamily_idを特定するため	tokenが空でない	なし	T re
3	token hash化	生Tokenをhashへ変換する	DB検索用に必要のため	hash化成功	なし	T re
4	transaction開始	RefreshToken失効と監査ログを一体化する	logout済みなのに監査ログがない状態を防ぐため	Tx開始成功	TxManager.WithinTx	T
5	RefreshToken取得	token_hashに一致するTokenを取得する	family_idと所有者を確認するため	token_hashが存在する	RefreshToken.FindByTokenHashWithUser	D
6	所有者確認	TokenのUserIDと認証済みUserIDを比較する	他人のTokenを失効させないため	token.UserID == auth.UserID	なし	リ
7	family失効	同じfamily_idのRefreshTokenをrevoked_at付きにする	現在ログイン中のRefreshToken群を使えなくするため	family_idあり	RefreshToken.RevokeFamily	更
8	監査ログ作成	logout操作を記録する	認証操作の履歴を残すため	user_idあり	AuditLog.Create	リ
9	commit	DB変更を確定する	失効と監査ログが両方成功した	全処理成功	TxManager	c

順番	処理	何をしているのか	なぜ必要か	状態確認	Repository	
			場合のみ反映するため			
10	Cookie削除	refresh_token Cookieを削除する	クライアント側にもTokenを残さないため	commit済み	なし	なし

RefreshToken family全失効 / token_version増加条件

処理	条件	RefreshToken family全失効	user全refresh失効	token_version増加	理由	返すエラー / 結果
Signup成功	新規ユーザー登録成功	しない	しない	しない	まだRefreshTokenを発行していないため	201 Created
Login成功	email/password一致、UserStatusActive	しない	しない	しない	新しいRefreshToken familyを発行するだけ。既存セッションは強制終了しないため	200 OK
Login失敗	password不一致、未認証、存在しないemail	しない	しない	しない	ログインできていないため、既存セッションには触らない	401 / 403
Refresh成功	refresh tokenが有効、未使用、未失効、期限内	しない	しない	しない	正常なtoken rotation。古いtokenをused_at付きにし、新しいrefresh tokenを発行するだけ	200 OK
Refresh再利用検知	すでにused_atが入ったrefresh tokenが再送された	する	しない	する	盗まれたRefreshTokenが再利用された可能性が高いため、同一familyを全失効し、users.token_versionを増加させて既存AccessTokenを無効化する。user全RefreshToken失効はLogoutAllDevicesで扱う	401 Unauthorized
Refresh失効済みtoken	revoked_atが入ったrefresh tokenが使われた	する	しない	する	失効済みtokenの再利用疑いとして、同一familyを全失効し、users.token_versionを増加させて既存AccessTokenを無効化する。user全RefreshToken失効はLogoutAllDevicesで扱う	401 Unauthorized
Refresh期限切れ	expires_at < now	しない	しない	しない	期限切れは通常の認証切れとして扱う。攻撃確定ではないため強制全失効しない	401 Unauthorized
Refresh token不正	token形式不正、hash照合不可、DBに存在しない	しない	しない	しない	familyを特定できないため全失効できない。token_versionも増やさない	401 Unauthorized

処理	条件	RefreshToken family全失効	user全refresh失効	token_version増加	理由	返すエラー / 結果
LogoutCurrentFamily成功	現在使用中のRefreshToken familyのみを失効する	する	しない	しない	通常ログアウトは現在の端末・現在のRefreshToken familyだけを無効化する処理であり、他端末のAccessTokenやRefreshTokenまで無効化する処理ではないため。全端末ログアウト、RefreshToken再利用検知、パスワードリセット後の強制ログアウトではusers.token_versionを更新する	204 No Content
Logout時にrefresh tokenなし	Cookieなし、tokenなし	しない	しない	しない	対象familyを特定できないためDB更新は行わない。ControllerはRefreshToken Cookie / CSRF Cookieを削除し、冪等なlogoutとして成功扱いにする。	204 No Content
LogoutAllDevices成功	認証済みUserが全端末ログアウトを実行	する	する	する	Userの全refresh tokenを失効し、既存AccessTokenも無効化するため	204 No Content
Me取得	access token有効、token_version一致	しない	しない	しない	参照処理のみ。refresh tokenには触らない	200 OK
Me取得失敗	access tokenのtoken_versionがDBの値と不一致	しない	しない	しない	すでに無効化済みのaccess tokenなので、追加処理は不要	401 Unauthorized

PasswordReset の扱い

現時点の実装対象には含めない。

対象	実装
ForgotPassword API	しない
ResetPassword API	しない
password_reset_tokens table	作らない
PasswordResetUsecase	作らない
PasswordResetValidator	作らない
PasswordResetTokenRepository	作らない
ForgotPasswordController / ResetPasswordController	作らない
/auth/forgot-password /auth/reset-password	作らない
forgot-password / reset-password RateLimit	作らない

そのため、現行のtransaction一覧、Repository呼び出し順、token_version更新条件には PasswordReset を含めない。

将来実装する場合は、以下を追加してから扱う。

追加対象	内容
Entity	PasswordResetToken

追加対象	内容
DB	password_reset_tokens
Repository	PasswordResetTokenRepository
Usecase	ForgotPassword / ResetPassword
Validator	email / reset_token / new_password
Controller	ForgotPasswordController / ResetPasswordController
Router	/auth/forgot-password, /auth/reset-password
RateLimit	forgot-password / reset-password
Security	token hash保存、期限、使用済み管理、使用済み後の再利用禁止、全RefreshToken失効、token_version更新

RefreshToken family / token_version ルール

ルール	内容
RefreshToken family全失効の対象	同じ <code>family_id</code> を持つRefreshTokenをすべて <code>revoked_at = now</code> にする
user全refresh失効の対象	同じ <code>user_id</code> を持つRefreshTokenをすべて <code>revoked_at = now</code> にする
token_version増加の対象	<code>users.token_version</code> を <code>+1</code> する
token_versionの目的	既存のAccessTokenを即時無効化するため
access token検証時の条件	JWT内の <code>token_version</code> とDBの <code>users.token_version</code> が一致すること
Refresh成功時の処理	古いRefreshTokenに <code>used_at</code> と <code>replaced_by</code> を入れ、新しいRefreshTokenを同じ <code>family_id</code> で作成する
reuse検知時の処理	<code>RevokeFamily(family_id)</code> → <code>IncrementTokenVersion(user_id)</code> → <code>401 Unauthorized</code> を返す
LogoutCurrentFamily成功時の処理	<code>RevokeFamily(family_id)</code> → Cookie削除
LogoutAllDevices成功時の処理	<code>RevokeByUserID(user_id)</code> → <code>IncrementTokenVersion(user_id)</code> → Cookie削除

GuestSessionUsecaseGetOrCreate

順番	処理	何をしているか	Repository呼び出し	説明	主なエラー
1	session key確認	Cookieにguest session keyがあるか確認する	なし	Cookie操作自体はControllerまたはmiddleware	なし
2	hash化	session keyをhash化する	なし	生値はDB保存しない	<code>ErrInvalidS</code>
3	既存session取得	hashでGuestSessionを探す	<code>GuestSessionRepository.FindBySessionKeyHash</code>	存在すれば再利用	<code>ErrNotFound</code>
4	期限確認	<code>expires_at</code> を確認する	なし	期限切れなら新規作成扱い	<code>ErrGuestSes</code>
5	最終アクセス更新	<code>last_seen_at</code> と <code>expires_at</code> を延長する	<code>GuestSessionRepository.Touch</code>	継続利用を反映	<code>ErrUpdateFa</code>
6	新規作成	sessionがなければ新規発行する	<code>GuestSessionRepository.Create</code>	hashのみ保存	<code>ErrCreateFa</code>
7	結果返却	GuestSessionと生session keyを返す	なし	Cookie設定はController	なし

BeanUsecaseListBeans

順番	処理	何をしているか	Repository呼び出し	説明	主なエラー
1	pagination確認	limit / offsetを確認する	なし	範囲外なら補正またはエラー	ErrInvalidPagination
2	公開Bean取得	公開中Beanだけ取得する	BeanRepository.ListPublished	非公開Beanは返さない	ErrRepositoryFailed
3	結果返却	一覧表示用データを返す	なし	Response DTO変換はController	なし

GetBeanDetail

順番	処理	何をしているか	Repository呼び出し	説明	主なエラー
1	Bean取得	公開中Beanを取得する	BeanRepository.FindPublishedByID	非公開Beanは not found 扱い	ErrBeanNotFound
2	関連記事取得	Beanに紐づくArticleを取得する	BeanArticleRepository.ListByBeanID	最大3件など、表示仕様に合わせて取得する	ErrRepositoryFailed
3	結果返却	Bean詳細と関連記事を返す	なし	content_view はここでは記録しない。表示イベントはEventUsecase.RecordEventに集約する	なし

ArticleUsecaseListArticles

順番	処理	何をしているか	Repository呼び出し	説明	主なエラー
1	pagination確認	limit / offsetを確認する	なし	範囲外を防ぐ	ErrInvalidPagination
2	公開Article取得	公開中Articleだけ取得する	ArticleRepository.ListPublished	下書きは返さない	ErrRepositoryFailed
3	結果返却	Article一覧を返す	なし	新着順など	なし

GetArticleDetail

Article本文を含む詳細取得は User / Admin のみ許可する。

GuestはArticle一覧・検索結果・summaryまでは閲覧できるが、本文詳細を取得しようとした場合はログイン要求エラーを返す。

順番	処理	何をしているか	Repository	主なエラー
1	actor確認	UserIDがあるか確認する。 GuestSessionIDのみの場合は詳細本文を返さない。	なし	ErrLoginRequired
2	slug確認	URLのslugを受け取る。	なし	ErrInvalidInput
3	公開Article取得	公開済みArticleをslugで取得する。	ArticleRepository.FindPublishedBySlug	ErrArticleNotFound
4	関連Bean取得	Articleに紐づく関連Beanを取得する。	BeanArticleRepository.ListByArticleID	ErrRepositoryFailed
5	結果返却	Article本文、summary、関連Beanを返す。	なし	なし

SearchUsecaseSearchBeans

順番	処理	何をしているか	Repository呼び出し	説明	主なエラー
1	actor確認	UserIDまたはGuestSessionIDの片方だけがあるか確認する	なし	両方nil、両方ありは禁止	ErrActorRequired
2	検索条件確認	keyword, origin, roast_level, 味覚条	なし	Validatorで形式検証済み。Usecase	ErrInvalidSearchCondition

順番	処理	何をしているか	Repository呼び出し	説明	主なエラー
		件, sort, limit, offsetを確認する		では業務上の範囲を最終確認する	
3	検索条件正規化	空白、大小文字、nil、デフォルト値を揃える	なし	Repositoryへ渡す前に条件の揺れをなくす	なし
4	Bean検索	条件に合う公開Beanを取得する	BeanRepository.SearchPublished	公開中のみ対象。sortに従って並べる	ErrRepositoryFailed
5	結果返却	検索結果を返す	なし	件数0でも正常	なし

SearchArticles

順番	処理	何をしているか	Repository呼び出し	説明	主なエラー
1	actor確認	UserIDまたはGuestSessionIDの片方だけがあるか確認する	なし	両方nil、両方ありは禁止	ErrActorRequired
2	検索条件確認	keyword, category, sort, limit, offsetを確認する	なし	tagはDB設計がないため扱わない	ErrInvalidSearchCondition
3	検索条件正規化	空白、大小文字、nil、デフォルト値を揃える	なし	Repositoryへ渡す前に条件の揺れをなくす	なし
4	Article検索	条件に合う公開Articleを取得する	ArticleRepository.SearchPublished	公開中のみ対象。sortに従って並べる	ErrRepositoryFailed
5	結果返却	検索結果を返す	なし	0件も正常	なし

EventUsecaseRecordEvent

- RecordEvent は POST /events から来る行動イベントを記録する。
- 直接受け付ける event_type は以下に限定する。

event_type	内容
content_view	コンテンツ詳細を閲覧した
impression	コンテンツが画面内に表示された
stay	一定時間滞在した
click	通常クリックした
re_search	検索条件を変更して再検索した

以下のevent_typeは POST /events では直接受け付けない。

event_type	担当Usecase	理由
save	SavedItemUsecase	保存/解除の業務ルールがあるため
rating	RatingUsecase	1ユーザー1評価、再評価、Good/Badの業務ルールがあるため
modal_impression	ModalUsecase	モーダル表示可否・抑制ルールがあるため
modal_click	ModalUsecase	ModalDisplayLogとの本人確認が必要なため
modal_close	ModalUsecase	closed_at更新とRedis抑制処理が必要なため

共通処理

順番	処理	何をしているのか	Repository	主なエラー
1	actor確認	UserIDまたはGuestSessionIDの片方だけがあることを確認する。両方nil・両方ありは禁止。	なし	ErrActorRequired

順番	処理	何をしているのか	Repository	主なエラー
2	event_type確認	content_view / impression / stay / click / re_search のみ許可する。	なし	<code>ErrInvalidEventType</code>
3	発生時刻確認	occurred_at が未来すぎないことを確認し、未指定なら now を使う。	なし	<code>ErrInvalidInput</code>

content_view / impression / click

順番	処理	何をしているのか	Repository	主なエラー
4	rank_target確認	rank_target_id があり、有効なランキング対象か確認する。	<code>RankTargetRepository.ExistsActiveByID</code>	<code>ErrRankTargetNotFound</code>
5	dedup判定	content_view / impression は短時間重複を防ぐ。click は必要な場合のみdedupする。	<code>EventDedupRepository.SetIfNotExists</code>	<code>ErrDuplicateEvent</code> / <code>ErrRedisFailed</code>
6	イベント作成	ActionEventを作成する。 rating_score, search_condition_hash, modal_display_log_id は nil。	<code>ActionEventRepository.Create</code>	<code>ErrEventRecordFailed</code>
7	結果返却	作成した場合は Recorded=true、重複なら Recorded=false を返す。	なし	なし

stay

順番	処理	何をしているのか	Repository	主なエラー
4	rank_target確認	rank_target_id があり、有効なランキング対象か確認する。	<code>RankTargetRepository.ExistsActiveByID</code>	<code>ErrRankTargetNotFound</code>
5	滞在時間確認	dwel_ms は 3000～1800000 の範囲だけ許可する。	なし	<code>ErrStayTooShort</code> / <code>ErrInvalidInput</code>
6	イベント作成	event_type=stay、dwel_ms付きで ActionEventを作成する。	<code>ActionEventRepository.Create</code>	<code>ErrEventRecordFailed</code>
7	結果返却	Recorded=true を返す。	なし	なし

re_search

順番	処理	何をしているのか	Repository	主なエラー
4	検索条件確認	search_target, q, category, origin, roast_level, acidity, bitterness, flavor, aroma, body を確認する。	なし	<code>ErrInvalidSearchCondition</code>
5	検索条件正規化	空白、大小文字、未指定値を揃える。	なし	なし
6	hash作成	正規化済み検索条件から condition_hash を作る。clientから hashは受け取らない。	なし	<code>ErrHashFailed</code>
7	前回hash取得	同一actorの直近 re_search hash を取得する。	<code>ActionEventRepository.FindLastSearchHash</code>	<code>ErrRepositoryFailed</code>
8	重複判定	今回hashと前回hashが同じなら	なし	なし

順番	処理	何をしているのか	Repository	主なエラー
		Recorded=false で返す。		
9	イベント作成	rank_target_id=nil、検索条件snapshot付きでActionEventを作成する。	ActionEventRepository.Create	ErrEventRecordFailed
10	結果返却	記録結果を返す。	なし	なし

SavedItemUseCaseSave

ユーザーの保存操作とランキング材料になるeventを一体化

順番	処理	何をしているのか	Repository呼び出し	説明	主なエラー
1	User確認	GuestではなくUserか確認する	なし	Guest保存不可	ErrLoginRequired
2	transaction開始	保存状態更新とsaveイベント作成を1つのtransactionにする	TxManager.WithinTx	部分成功を防ぐ	ErrTransactionFai
3	RankTarget確認	保存対象が有効か確認する	RankTargetRepository.ExistsActiveByID	非公開・無効対象は保存不可	ErrRankTargetNotF
4	保存または復元	未保存なら作成、解除済みなら復元	SavedItemRepository.SaveOrRestore	重複保存を防ぐ	ErrSaveFailed
5	saveイベント記録	保存成功をActionEventに残す	ActionEventRepository.Create	rankingに反映する	ErrEventRecordFai
6	transaction完了	commitまたはrollbackする	TxManager.WithinTx	イベント失敗時は保存もrollback	ErrTransactionFai
7	結果返却	保存状態を返す	なし	すでに保存済みならその状態を返す	なし

TX

順番	処理	何をしているのか	なぜ必要か	状態確認	Repository	rollback
1	認証User確認	user_idを取得する	保存はログインユーザー専用機能のため	user_idあり	なし	Tx開 rollback
2	入力確認	rank_target_idなどを確認する	保存対象を特定するため	IDが正の値	なし	Tx開 rollback
3	transaction開始	保存状態とeventを一体化する	保存だけ成功してeventがない状態を防ぐため	Tx開始成功	TxManager.WithinTx	Tx開
4	RankTarget確認	保存対象が存在し有効か確認する	非公開・削除済み対象を保存させないため	rank_targets.is_active = true	RankTarget.ExistsActiveByID	存在 inac
5	既存保存確認	すでに保存済みか、削除済み保存かを確認する	二重保存を防ぎ、解除済みなら復元するため	user_id + rank_target_id	SavedItem.FindByUserAndTarget	DB引
6	保存または復元	未保存なら作成、removed_atありなら復元する	1ユーザー1対象1保存にするため	unique制約に違反しない	SavedItem.SaveOrRestore	作成 失敗

順番	処理	何をしているのか	なぜ必要か	状態確認	Repository	rollback
7	save event作成	action_eventsへsave操作を記録する	ランキング計算・興味プロフィール更新に使うため	event_type = save	ActionEvent.Create	作成
8	commit	保存とeventを確定する	両方成功した場合のみ保存扱いにするため	全処理成功	TxManager	commit
9	結果返却	保存済み状態を返す	フロントでボタン状態を更新するため	commit済み	なし	なし

Remove

順番	処理	何をしているのか	Repository呼び出し	説明	主なエラー
1	User確認	保存解除するUserを確認する	なし	Guest不可	ErrLoginRequired
2	保存解除	removed_atを設定する	SavedItemRepository.Remove	物理削除ではない	ErrSavedItemNotFound, ErrRemoveFailed
3	結果返却	解除成功を返す	なし	removeイベントは必要なら追加	なし

RatingUsecaseRate

順番	処理	何をしているのか	Repository呼び出し	説明	主なエラー
1	User確認	GuestではなくUserか確認する	なし	Guest評価不可	ErrLoginRequired
2	score確認	+1または-1か確認する	なし	0, 2, 5 は不可	ErrInvalidRatingS
3	transaction開始	評価保存とratingイベント作成を1つのtransactionにする	TxManager.WithinTx	部分成功を防ぐ	ErrTransactionFai
4	RankTarget確認	評価対象が有効か確認する	RankTargetRepository.ExistsActiveByID	無効対象は評価不可	ErrRankTargetNotF
5	評価保存	初回評価または再評価を行う	RatingRepository.Upsert	1ユーザー1対象1評価	ErrRatingFailed
6	ratingイベント記録	評価成功をActionEventに残す	ActionEventRepository.Create	rating_scoreに保存	ErrEventRecordFai
7	transaction完了	commitまたはrollbackする	TxManager.WithinTx	イベント失敗時は評価もrollback	ErrTransactionFai
8	結果返却	現在の評価を返す	なし	画面の状態更新用	なし

TX

順番	処理	何をしているのか	なぜ必要か	状態確認	Repository	rollback
1	認証User確認	user_idを取得する	評価はログインユーザー専用機能のため	user_idあり	なし	Tx開始前rollback

順番	処理	何をしているのか	なぜ必要か	状態確認	Repository	rollback
2	入力確認	rank_target_idとscoreを確認する	評価対象と評価値を確定するため	scoreは +1 または -1	なし	Tx開始前rollback
3	transaction開始	評価保存とeventを一体化する	評価だけ成功してeventがない状態を防ぐため	Tx開始成功	TxManager.WithinTx	Tx開始失敗
4	RankTarget確認	評価対象が存在し有効か確認する	非公開・削除済み対象を評価させないため	rank_targets.is_active = true	RankTarget.ExistsActiveByID	存在なしinactive
5	既存評価確認	すでに評価済みか確認する	1ユーザー1対象1評価にするため	user_id + rank_target_id	Rating.FindByUserAndTarget	DB失敗
6	評価upsert	未評価なら作成、評価済みなら更新する	再評価できる仕様にするため	unique制約に違反しない	Rating.Upsert	作成失敗失敗
7	rating event作成	action_eventsへ評価操作を記録する	ランキング計算・興味プロフィール更新に使うため	event_type = rating	ActionEvent.Create	作成失敗
8	commit	評価とeventを確定する	両方成功した場合のみ評価完了にするため	全処理成功	TxManager	commit失敗
9	結果返却	現在の評価値を返す	フロントでGood/Bad状態を更新するため	commit済み	なし	なし

RecommendationUseCaseListRecommendations

順番	処理	何をしているのか	Repository呼び出し	説明	主なエラー
1	actor確認	UserまたはGuestSessionを確認する	なし	ログイン中ならUser、未ログインならGuestSessionを使う	EntityNotFoundException
2	contentType確認	Bean推薦かArticle推薦か総合推薦かを確認する	なし	bean / article / nil を許可する	EntityNotFoundException
6	興味スコア取得	User / Guestの興味プロフィールを取得する	InterestProfileRepository.ListTopByUser または ListTopByGuest	産地、焙煎度、風味、記事カテゴリなどの好みを取得する	EntityNotFoundException
7	metrics取得	scoreの高いContentMetricを取得する	ContentMetricRepository.ListRanking または ListTopByScore	ランキング計算済みの候補を取得する	EntityNotFoundException
8	RankTarget取得	metricsに紐づくRankTargetを取得する	RankTargetRepository.FindByIDs	content_type と content_id を確認する	EntityNotFoundException
9	保存済み除外	Userの場合、保存済み候補を除外する	SavedItemRepository.ExistsActive	既に保存したものを推薦しすぎないため	EntityNotFoundException
10	実体取得	Bean / Article本体を取得する	BeanRepository.FindByIDs / ArticleRepository.FindByIDs	表示に必要な情報を取得する	EntityNotFoundException
11	推薦スコア補正	ranking scoreに興味スコアを加味する	なし	User / Guestごとに少し順位を変える	EntityNotFoundException
12	推薦理由作成	なぜおすすめするのかを文章化	なし	例:「浅煎り・Ethiopiaへの反応が多いため」	EntityNotFoundException

		する			
13	score順並び替え	最終score順に並べる	なし	SortRecommendationsByScore	なし
15	結果返却	推薦一覧を返す	なし	件数0でも正常	なし

BuildRecommendationReason

順番	処理	何をしているか	Repository呼び出し	説明	主なエラー
1	候補確認	推薦対象のBean / Article情報を受け取る	なし	表示対象の情報を使う	ErrInvalidInput
2	興味軸確認	User / Guestの興味スコアを確認する	なし	origin、roast_level、flavor、categoryなど	なし
3	metrics確認	click率、save率、Good率などを確認する	なし	人気・反応の強さを理由に使う	なし
4	理由生成	ユーザー向け推薦理由を作る	なし	DB保存はしない。レスポンス用	ErrRecommendationReasonFailed
5	結果返却	reason文字列を返す	なし	例: 「保存率が高く、浅煎り好きに合いやすい豆です」	なし

NormalizeSearchCondition

順番	処理	何をしているか	Repository呼び出し	説明	主なエラー
1	入力受け取り	検索条件を受け取る	なし	Controller DTOからUsecase inputへ変換済み	ErrInvalidInput
2	空白除去	keywordの前後空白を削除する	なし	hashの揺れを防ぐ	なし
3	nil整理	空文字をnilに寄せる	なし	同じ意味の条件を同一扱いする	なし
4	enum確認	content_type、roast_levelなどを確認する	なし	許可値以外を弾く	ErrInvalidSearchCondition
5	数値範囲確認	acidity、bitterness、flavorなどを確認する	なし	1〜5のみ	ErrInvalidSearchCondition
6	正規化結果返却	hash化できる形で返す	なし	BuildConditionHash に渡す	なし

BuildConditionHash

順番	処理	何をしているか	Repository呼び出し	説明	主なエラー
1	正規化済み条件受け取り	Normalize済み条件を受け取る	なし	未正規化のままhash化しない	ErrInvalidInput
2	key順に並べる	条件の順序を固定する	なし	JSON化や文字列化の順序を固定する	なし
3	hash生成	sha256等でhashを作る	なし	DBには検索条件hashを保存する	ErrHashFailed
4	結果返却	hash文字列を返す	なし	ActionEvent.SearchConditionHash に使う	なし

RankingUsecaseListRanking

順番	処理	何をしているか	Repository呼び出し	説明	主なエラー
1	contentType確認	BeanランキングかArticleランキングか確認する	なし	nilなら総合ランキングも可	<code>ErrInvalidContentTy</code>
2	metrics取得	score順にContentMetricを取得する	<code>ContentMetricRepository.ListRanking</code>	日次計算済みの結果を読む	<code>ErrRepositoryFailed</code>
3	RankTarget整理	metricsのrank_target_idを集める	なし	実体取得の準備	なし
4	RankTarget取得	対応するRankTargetを取得する	<code>RankTargetRepository.FindByIds</code>	content_type / content_idを見る	<code>ErrRepositoryFailed</code>
5	実体取得	BeanまたはArticleを取得する	<code>BeanRepository.FindByIds</code> / <code>ArticleRepository.FindByIds</code>	content_typeごとに分けて取得	<code>ErrRepositoryFailed</code>
6	表示結果作成	metric + 実体情報を結合する	なし	DTO化はControllerでもよい	<code>ErrRankingBuildFail</code>
7	結果返却	ランキング一覧を返す	なし	score降順	なし

ModalUsecaseCanShow / ShowModal

順番	処理	何をしているか	Repository呼び出し	説明
1	actor確認	UserまたはGuestSessionを確認する	なし	モーダル制御はactor単位
2	トリガー確認	modal表示のきっかけを確認する	なし	enum以外不可
3	session上限確認	同一セッションで出しすぎているか確認する	<code>ModalSuppressionRepository.IncrementSessionCount</code> または <code>ModalDisplayLogRepository.CountShownInSession</code>	最大1回など
4	page上限確認	同一ページで出しすぎているか確認する	<code>ModalSuppressionRepository.IncrementPageCount</code> または <code>ModalDisplayLogRepository.CountShownOnPage</code>	1ページ1回
5	興味スコア取得	User / Guestの興味を取得する	<code>InterestProfileRepository.ListTopByUser</code> / <code>ListTopByGuest</code>	候補選定の材料
6	候補取得	ranking上位または興味に近い候補を取得する	<code>ContentMetricRepository.ListTopByScore</code>	初期段階はランキング上位でもよい
7	保存済み除外	既に保存済みの候補を外す	<code>SavedItemRepository.ExistsActive</code>	Userのみ
8	最近表示確認	同じ候補を最近出していないか確認する	<code>ModalSuppressionRepository.WasShown</code> / <code>ModalDisplayLogRepository.WasTargetShownRecently</code>	24時間抑制
9	最近閉じた確認	直近で閉じた候補を避ける	<code>ModalSuppressionRepository.WasClosed</code> / <code>ModalDisplayLogRepository.WasTargetClosedRecently</code>	興味が低い可能性
10	表示ログ作成	表示した事実を保存する	<code>ModalDisplayLogRepository.Create</code>	modal_display_logs
11	impression記録	modal_impressionをActionEventに保存する	<code>ActionEventRepository.Create</code>	modal CTR計算用

12	Redis抑制保存	表示済み候補としてRedisに保存する	<code>ModalSuppressionRepository.SetShown</code>	24時間抑制	
13	結果返却	modal表示対象を返す	なし	画面に表示する	7

ClickModal

ClickModalは、本人または本人GuestSessionのmodal_display_logだけをクリック済みにする。
modal_display_log_idだけで更新すると、他人のログを操作できるため禁止。

順番	処理	何をしているか	Repository呼び出し	説明	主なユースケース
1	Actor確認	UserIDまたはGuestSessionIDの片方だけがあることを確認	なし	両方nil、両方ありは禁止	ErrAr
2	transaction開始	表示ログ更新とイベント作成をまとめる	<code>TxManager.WithinTx</code>	部分成功を防ぐ	ErrTi
3	表示ログ確認	modal_display_log_idが本人のものが確認する	<code>ModalDisplayLogRepository.FindByIDForActor</code>	他人のログ操作を防ぐ	ErrMk
4	クリック日時更新	clicked_atを設定する	<code>ModalDisplayLogRepository.MarkClickedForActor</code>	モーダルCTR計算に使う	ErrUj
5	modal_click記録	ActionEventにクリックを保存する	<code>ActionEventRepository.Create</code>	通常clickとは分ける	ErrEv
6	結果返却	遷移先情報を返す	なし	Bean / Article詳細へ遷移	なし

Tx

順番	処理	何をしているのか	なぜ必要か	状態確認	Repository
1	Actor確認	user_idまたはguest_session_idを取得する	誰のモーダル操作か特定するため	user_idまたはguest_session_idの片方だけ存在	なし
2	入力確認	modal_display_log_idを確認する	どの表示ログへのクリックか特定するため	IDが正の値	なし
3	transaction開始	clicked_at更新とevent作成を一体化する	click状態とeventがズレるのを防ぐため	Tx開始成功	TxManager.WithinTx
4	表示ログ取得	modal_display_log_idのログを取得する	実在する表示ログが確認するため	logが存在	ModalDisplayLog.FindByIDFor
5	本人確認	ログのuser_id / guest_session_idがactorと一致するか確認する	他人の表示ログをクリック扱いにされるのを防ぐため	actor一致	なし
6	クリック済み確認	すでにclicked_atが入っていないか確認する	二重クリック集計を防ぐため	clicked_at == nil	なし
7	clicked_at更新	modal_display_logs.clicked_atを更新する	モーダルがクリックされた事実を保存するため	対象logが本人のもの	ModalDisplayLog.MarkClicked
8	modal_click event作成	action_eventsへmodal_clickを記録する	CTR計算、推薦改善に使うため	event_type = modal_click	ActionEvent.Create
9	commit	clicked_atとeventを確定する	両方成功した場合のみclick扱いにするため	全処理成功	TxManager
10	結果返却	クリック成功を返す	フロントが遷移や表示更新を行	commit済み	なし

順番	処理	何をしているのか	なぜ必要か	状態確認	Repository
			うため		

CloseModal

CloseModalは、本人または本人GuestSessionのmodal_display_logだけを閉じた状態にする。
modal_display_log_idだけで更新すると、他人のログを操作できるため禁止。

順番	処理	何をしているのか	Repository呼び出し	説明	主なエラー
1	Actor確認	UserIDまたはGuestSessionIDの片方だけがあることを確認	なし	両方nil、両方ありは禁止	ErrActor
2	transaction開始	表示ログ更新、抑制保存、イベント作成をまとめる	<code>TxManager.WithinTx</code>	部分成功を防ぐ	ErrTransaction
3	表示ログ確認	modal_display_log_idが本人のものか確認する	<code>ModalDisplayLogRepository.FindByIDForActor</code>	他人のログ操作を防ぐ	ErrModalDisplayLog
4	閉じた日時更新	closed_atを設定する	<code>ModalDisplayLogRepository.MarkClosedForActor</code>	閉じ率計算に使う	ErrUpdate
5	Redis抑制保存	閉じた候補を一定時間出さない	<code>ModalSuppressionRepository.SetClosed</code>	しつこい表示防止	ErrRedis
6	modal_close記録	ActionEventにcloseを保存する	<code>ActionEventRepository.Create</code>	推薦不一致の材料	ErrEvent
7	結果返却	close成功を返す	なし	なし	なし

Tx

順番	処理	何をしているのか	なぜ必要か	状態確認	Repository
1	Actor確認	user_idまたはguest_session_idを取得する	誰がモーダルを閉じたか特定するため	user_idまたはguest_session_idの片方だけ存在	なし
2	入力確認	modal_display_log_idを確認する	どの表示ログをclose扱いにするか特定するため	IDが正の値	なし
3	transaction開始	closed_at更新とevent作成を一体化する	close状態とeventがズレるのを防ぐため	Tx開始成功	TxManager.WithinTx
4	表示ログ取得	modal_display_log_idのログを取得する	実在する表示ログか確認するため	logが存在	ModalDisplayLog.FindByIDFor
5	本人確認	ログのuser_id / guest_session_idがactorと一致するか確認する	他人の表示ログをclose扱いにされるのを防ぐため	actor一致	なし
6	close済み確認	すでにclosed_atが入っていないか確認する	二重close集計を防ぐため	closed_at == nil	なし
7	closed_at更新	modal_display_logs.closed_atを更新する	モーダルを閉じた事実を保存するため	対象logが本人のもの	ModalDisplayLog.MarkClosed
8	modal_close event作成	action_eventsへmodal_closeを記録する	モーダル改善・抑制判定の材料にするため	event_type = modal_close	ActionEvent.Create

順番	処理	何をしているのか	なぜ必要か	状態確認	Repository
9	commit	closed_atとeventを確定する	両方成功した場合のみclose扱いにするため	全処理成功	TxManager
10	閉じたモーダルの再表示防止を保存	一定時間、同じモーダルを出しにくくする	ユーザー体験を悪化させないため	commit済み	ModalSuppression.SetClosed
11	結果返却	close成功を返す	フロントがモーダルを閉じるため	commit済み	なし

RankingBatchUsecase

`RankingBatchUsecase` は、各 `RankTarget` の行動集計値をもとに `ranking score` を算出し、`content_metrics.score` へ保存する。

`score` は0～100の範囲とし、閲覧、クリック、保存、Good評価、滞在時間、モーダルクリックを加算対象とする。

Bad評価、モーダル即閉じは減点対象とする。

`re_search` は検索条件改善・興味分析には使うが、`content_metrics.score` には使わない。

理由は、`re_search` は検索条件変更イベントであり、特定の `RankTarget` に直接紐づかないため。

```

score =
  10
+ min(log(1 + content_view_count) * 4, 16)
+ min(click_count / max(impression_count, 1) * 20, 20)
+ min(save_count / max(content_view_count, 1) * 25, 25)
+ min(good_count / max(good_count + bad_count, 1) * 15, 15)
- min(bad_count / max(good_count + bad_count, 1) * 20, 20)
+ min((stay_total_ms / max(content_view_count, 1)) / 30000 * 10, 10)
+ min(modal_click_count / max(modal_impression_count, 1) * 8, 8)
- min(modal_close_count / max(modal_impression_count, 1) * 8, 8)

final_score = min(max(score, 0), 100)

```

Recalculate

順番	処理	何をしているのか	Repository呼び出し	説明	主なエラー
1	lock取得	バッチ二重実行を防ぐ	<code>BatchLockRepository.Acquire</code>	既に実行中なら中止	<code>ErrBatchLock</code>
2	実行ログ作成	BatchRunをrunningで作成する	<code>BatchRunRepository.Create</code>	実行履歴を残す	<code>ErrBatchRunCreate</code>
3	集計期間決定	直近30日などを決める	なし	<code>RankingWindowDays</code>	なし
4	行動ログ集計	action_eventsを集計する	<code>ActionEventRepository.AggregateContentMetrics</code>	content_metricsの材料	<code>ErrAggregateMetrics</code>
5	score計算	CTR、保存率、評価平均などからscoreを計算する	なし	業務ロジック。Repositoryでは計算しない	<code>ErrScoreCalculation</code>
6	metrics保存	集計結果をupsertする	<code>ContentMetricRepository.BulkUpsert</code>	1RankTargetにつき1行	<code>ErrBulkUpsert</code>
7	成功更新	BatchRunをsuccessにする	<code>BatchRunRepository.MarkSuccess</code>	rows_processedを保存	<code>ErrBatchRunMarkSuccess</code>
8	監査ログ作成	バッチ実行を記録する	<code>AuditLogRepository.Create</code>	systemまたはadmin	<code>ErrAuditLogCreate</code>

9	lock解放	Redis lockを解放する	<code>BatchLockRepository.Release</code>	deferで実行	<code>ErrRedisFailed</code>
10	結果返却	処理件数などを返す	なし	Admin画面にも表示できる	なし

失敗時

順番	処理	何をしているか	Repository呼び出し	説明	主なエラー
1	エラー捕捉	途中で失敗した内容を拾う	なし	panicではなくerrorで扱う	各種error
2	失敗更新	BatchRunをfailedにする	<code>BatchRunRepository.MarkFailed</code>	error_messageを保存	<code>ErrUpdateFailed</code>
3	lock解放	実行ロックを解除する	<code>BatchLockRepository.Release</code>	必ず実行	<code>ErrRedisFailed</code>
4	エラー返却	usecase errorとして返す	なし	ControllerがHTTPに変換	元エラー

InterestBatchUsecaseRecalculateUserInterest

順番	処理	何をしているか	Repository呼び出し	説明	主なエラー
1	User確認	対象UserIDを確認する	なし	User単位で再計算	<code>ErrInvalidId</code>
2	集計期間決定	直近期間を決める	なし	30日など	なし
3	行動集計	Userの行動を興味軸ごとに集計する	<code>ActionEventRepository.AggregateUserInterest</code>	origin, roast_level, flavorなど	<code>ErrAggregate</code>
4	score変換	行動重みをScoreDeltaへ変換する	なし	save, rating, stayなどを重み付け	<code>ErrScoreCalc</code>
5	興味スコア保存	InterestProfileをupsertする	<code>InterestProfileRepository.BulkUpsert</code>	同じdimension/valueは更新	<code>ErrUpsertFailed</code>
6	結果返却	更新件数を返す	なし	なし	なし

RecalculateGuestInterest

順番	処理	何をしているか	Repository呼び出し	説明	主なエラー
1	Guest確認	GuestSessionIDを確認する	なし	Guest単位で再計算	<code>ErrInvalidId</code>
2	行動集計	Guestの行動を集計する	<code>ActionEventRepository.AggregateGuestInterest</code>	未ログインでも興味推定	<code>ErrAggregate</code>
3	expires_at設定	Guest用InterestProfileに期限を持たせる	なし	GuestSession期限と合わせる	なし
4	興味スコア保存	InterestProfileをupsertする	<code>InterestProfileRepository.BulkUpsert</code>	GuestSession単位	<code>ErrUpsertFailed</code>
5	結果返却	更新件数を返す	なし	なし	なし

AdminBeanUsecaseCreateBean

順番	処理	何をしているか	Repository呼び出し	説明	主なエラー
1	Admin確認	実行者がAdminか確認する	なし	Controller/middlewareで認証済みでも	<code>ErrForbidden</code>

				Usecaseで権限前提を持つ	
2	Bean作成	BeanをDBに保存する	<code>BeanRepository.Create</code>	初期状態は非公開でもよい	<code>ErrCreateFailed</code>
3	RankTarget作成	Bean用RankTargetを作る	<code>RankTargetRepository.FindOrCreate</code>	ranking対象を確保	<code>ErrCreateFailed</code>
4	監査ログ作成	bean_createを記録する	<code>AuditLogRepository.Create</code>	Admin操作の履歴	<code>ErrAuditLogFailed</code>
5	結果返却	作成したBeanを返す	なし	なし	なし

PublishBean / UnpublishBean

順番	処理	何をしているか	Repository呼び出し	説明	主なエラー
1	Admin確認	管理者操作か確認する	なし	Adminのみ	<code>ErrForbidden</code>
2	Bean存在確認	対象Beanが存在するか確認する	<code>BeanRepository.ExistsByID</code>	存在しないならエラー	<code>ErrBeanNotFound</code>
3	公開状態更新	is_publishedを更新する	<code>BeanRepository.UpdatePublished</code>	true/false	<code>ErrUpdateFailed</code>
4	RankTarget取得	BeanのRankTargetを取得する	<code>RankTargetRepository.FindByContent</code>	ranking対象	<code>ErrRankTargetNotFound</code>
5	RankTarget有効更新	is_activeを更新する	<code>RankTargetRepository.UpdateActive</code>	公開ならactive、非公開ならinactive	<code>ErrUpdateFailed</code>
6	監査ログ作成	publish/unpublishを記録する	<code>AuditLogRepository.Create</code>	Admin操作	<code>ErrAuditLogFailed</code>
7	結果返却	更新結果を返す	なし	なし	なし

AdminArticleUsecaseCreateArticle

順番	処理	何をしているか	Repository呼び出し	説明	主なエラー
1	Admin確認	実行者がAdminか確認する	なし	Adminのみ	<code>ErrForbidden</code>
2	slug重複確認	同じslugが既に使われていないか確認する	<code>ArticleRepository.ExistsBySlug</code>	trueなら作成不可。DB unique違反の前に業務エラーとして扱う	<code>ErrSlugAlreadyExists</code>
3	Article作成	Articleを保存する	<code>ArticleRepository.Create</code>	下書きまたは公開状態	<code>ErrCreateFailed</code>
4	RankTarget作成	Article用RankTargetを作る	<code>RankTargetRepository.FindOrCreate</code>	ranking対象化	<code>ErrCreateFailed</code>
5	監査ログ作成	article_createを記録する	<code>AuditLogRepository.Create</code>	Admin操作	<code>ErrAuditLogFailed</code>
6	結果返却	作成したArticleを返す	なし	なし	なし

AdminRelationUsecaseCreateBeanArticleRelation

順番	処理	何をしているか	Repository呼び出し	説明	主なエラー
----	----	---------	----------------	----	-------

1	Admin確認	実行者がAdminか確認する	なし	関連付け操作は管理者のみ	ErrForbidden
2	Bean存在確認	関連元Beanが存在するか確認する	BeanRepository.ExistsByID	存在しないBeanには紐付けない	ErrBeanNotFound
3	Article存在確認	関連先Articleが存在するか確認する	ArticleRepository.ExistsByID	存在しないArticleには紐付けない	ErrArticleNotFound
4	重複確認	同じBeanとArticleの関連が既にないか確認する	BeanArticleRepository.Exists	複合一意制約違反を事前に防ぐ	ErrRelationAlreadyExists
5	関連作成	BeanArticleを作成する	BeanArticleRepository.Create	display_orderも保存する	ErrCreateFailed
6	監査ログ作成	関連作成を記録する	AuditLogRepository.Create	Admin操作の履歴	ErrAuditLogFailed
7	結果返却	作成した関連を返す	なし	管理画面に反映する	なし

DeleteBeanArticleRelation

順番	処理	何をしているか	Repository呼び出し	説明	主なエラー
1	Admin確認	実行者がAdminか確認する	なし	管理者のみ	ErrForbidden
2	関連存在確認	削除対象の関連が存在するか確認する	BeanArticleRepository.Exists	存在しないなら404扱い	ErrRelationNotFound
3	関連削除	BeanとArticleの関連を削除する	BeanArticleRepository.Delete	物理削除でよい	ErrDeleteFailed
4	監査ログ作成	関連削除を記録する	AuditLogRepository.Create	Admin操作の履歴	ErrAuditLogFailed
5	結果返却	削除成功を返す	なし	なし	なし

ReplaceBeanArticleRelations

順番	処理	何をしているか	Repository呼び出し	説明	主なエラー
1	Admin確認	実行者がAdminか確認する	なし	管理者のみ	ErrForbidden
2	Bean存在確認	対象Beanが存在するか確認する	BeanRepository.ExistsByID	関連元の存在確認	ErrBeanNotFound
3	Article件数確認	関連Article数が上限以内か確認する	なし	Bean詳細の関連記事は最大3件などに制限する	ErrTooManyRelatio
4	Article存在確認	指定されたArticleが全て存在するか確認する	ArticleRepository.ExistsByID	1件でも存在しなければ失敗	ErrArticleNotFoun
5	関連差し替え	既存関連を削除して新しい関連を作る	BeanArticleRepository.ReplaceByBeanID	transaction内で実行する	ErrUpdateFailed
6	監査ログ作成	関連差し替えを記録する	AuditLogRepository.Create	Admin操作	ErrAuditLogFailed
7	結果返却	更新後の関連一覧を返す	BeanArticleRepository.ListByBeanID	管理画面で確認できるようにする	ErrRepositoryFail

UpdateDisplayOrder

順番	処理	何をしているか	Repository呼び出し	説明	主なエラー
1	Admin確認	実行者がAdminか確認する	なし	管理者のみ	<code>ErrForbidden</code>
2	Bean存在確認	対象Beanが存在するか確認する	<code>BeanRepository.ExistsByID</code>	存在しないBeanの並び替えは不可	<code>ErrBeanNotFound</code>
3	Article存在確認	並び替え対象Articleを確認する	<code>ArticleRepository.ExistsByID</code>	存在しないArticleは不可	<code>ErrArticleNotFound</code>
4	関連存在確認	対象の関連が存在するか確認する	<code>BeanArticleRepository.Exists</code>	関連がなければ並び替え不可	<code>ErrRelationNotFound</code>
5	表示順更新	display_orderを更新する	<code>BeanArticleRepository.UpdateDisplayOrder</code>	小さい順に表示する	<code>ErrUpdateFailed</code>
6	監査ログ作成	表示順変更を記録する	<code>AuditLogRepository.Create</code>	Admin操作	<code>ErrAuditLogFailed</code>
7	結果返却	更新後の状態を返す	なし	なし	なし

AdminBatchUsecaseRunRankingBatch

順番	処理	何をしているか	呼び出し先	説明	主なエラー
1	Admin確認	実行者がAdminか確認する	なし	Adminのみ実行可能	<code>ErrForbidden</code>
2	実行中確認	既にrunningのbatchがあるか確認する	<code>BatchRunRepository.FindRunningByJobName</code>	二重実行防止	<code>ErrBatchAlreadyRunning</code>
3	バッチ実行	指定されたbatch種別に応じて専用Usecaseへ委譲する	<code>RankingBatchUsecase.Recalculate</code>	AdminBatchUsecaseには集計ロジックを書かない	<code>ErrAggregateScoreFailed</code> / <code>ErrUpdateScoreFailed</code>
4	監査ログ作成	manual_batch_runを記録する	<code>AuditLogRepository.Create</code>	Admin操作履歴として残す	<code>ErrAuditLogFailed</code>
5	結果返却	実行結果を返す	なし	処理件数、実行結果、エラー内容を返す	なし

AdminAuditUsecaseListAuditLogs

順番	処理	何をしているか	Repository呼び出し	説明	主なエラー
1	Admin確認	管理者か確認する	なし	Adminのみ	<code>ErrForbidden</code>
2	filter確認	actor, action, target, dateを確認する	なし	不正条件を弾く	<code>ErrInvalidFilter</code>
3	監査ログ検索	条件に合うログを取得する	<code>AuditLogRepository.List</code>	Admin画面用	<code>ErrRepositoryFailed</code>
4	結果返却	一覧を返す	なし	新しい順	なし

CleanupUsecaseDeleteExpired

順番	処理	何をしているか	Repository呼び出し	説明	主なエラー
1	現在時刻取得	削除基準時刻を決める	なし	<code>now</code>	なし
2	期限切れRefreshToken削除	期限切れtokenを削除する	<code>RefreshTokenRepository.DeleteExpired</code>	セキュリティ・容量対策	<code>ErrDeleteFailed</code>

3	期限切れ GuestSession削 除	期限切れ GuestSessionを 削除する	<code>GuestSessionRepository.DeleteExpired</code>	不要session削 除	<code>ErrDeleteFailed</code>
4	期限切れ InterestProfile 削除	Guest由来の期 限切れ興味を削 除する	<code>InterestProfileRepository.DeleteExpired</code>	不要興味デー タ削除	<code>ErrDeleteFailed</code>
5	結果返却	削除件数を返す	なし		

Repository呼び出し順まとめ

処理	Repository呼び出し順	何をしているか	transaction
<code>Signup</code>	User.ExistsByEmail → User.Create	新規Userを作成する。Signup成功時はAccessToken / RefreshTokenを発行しない	不要
<code>Login</code>	User.FindByEmail → RefreshToken.Create → AuditLog.Create(best effort)	認証成功後、RefreshTokenを発行する。login成功ログは失敗してもlogin本体を戻さない	不要
<code>Refresh</code>	TxManager.WithinTx(RefreshToken.FindByTokenHashWithUserForUpdate → RefreshToken.Create → RefreshToken.MarkUsed) → AuditLog.Create(best effort)	RefreshTokenを正常rotationし、新AccessTokenと新RefreshTokenを発行する	必須
<code>Refresh reuse検知</code>	TxManager.WithinTx(RefreshToken.RevokeFamily → User.IncrementTokenVersion) → AuditLog.Create(best effort)	使用済みRefreshTokenの再利用を検知し、関連tokenと既存AccessTokenを無効化する	必須
<code>LogoutCurrentFamily</code>	RefreshToken.FindByTokenHash → RefreshToken.RevokeFamily → AuditLog.Create(best effort)	現在端末のRefreshToken familyのみ失効する。token_versionは更新しない	不要
<code>LogoutAllDevices</code>	TxManager.WithinTx(RefreshToken.RevokeByUserID → User.IncrementTokenVersion) → AuditLog.Create(best effort)	対象userの全RefreshTokenと既存AccessTokenを失効する	必須
<code>Save</code>	RankTarget.ExistsActiveByID → SavedItem.SaveOrRestore → ActionEvent.Create(save / best effort)	保存対象が有効か確認し、保存または再保存する	不要
<code>RemoveSavedItem</code>	SavedItem.Remove	保存解除する	不要
<code>Rating</code>	RankTarget.ExistsActiveByID → Rating.Upsert → ActionEvent.Create(rating / best effort)	評価対象が有効か確認し、Good/Bad評価を作成または更新する	不要
<code>DeleteRating</code>	Rating.Delete	自分の評価を削除する	不要

処理	Repository呼び出し順	何をしているか	transaction
Event記録	EventDedup.CheckAndSet → RankTarget.ExistsActiveByID → ActionEvent.Create	フロントから送られた行動イベントを記録する。ActionEvent作成が本体なので失敗時はエラーにする	不要
Modal表示	ModalSuppression.Check → Candidate取得 → ModalDisplayLog.Create → ActionEvent.Create(modal_impression / best effort) → ModalSuppression.SetShown	モーダル表示可否を判定し、表示履歴を作成する	不要
Modalクリック	ModalDisplayLog.MarkClickedForActor → ActionEvent.Create(modal_click / best effort)	actor条件付きUPDATEで本人/GuestSessionの表示ログだけクリック済みにする	不要
Modal閉じる	ModalDisplayLog.MarkClosedForActor → ActionEvent.Create(modal_close / best effort) → ModalSuppression.SetClosed	actor条件付きUPDATEで本人/GuestSessionの表示ログだけ閉じる	不要
RankingBatch成功	BatchLock.Acquire → BatchRun.Create → ActionEvent.AggregateContentMetrics → TxManager.WithinTx(ContentMetric.BulkUpsert → BatchRun.MarkSuccess) → AuditLog.Create(best effort) → BatchLock.Release	行動ログを集計し、ランキング指標とBatchRun成功状態を保存する	保存部分のみ必須
RankingBatch失敗	BatchRun.MarkFailed → AuditLog.Create(best effort) → BatchLock.Release	失敗理由を保存する	不要
InterestBatch成功	BatchLock.Acquire → BatchRun.Create → ActionEvent.AggregateUserInterest / AggregateGuestInterest → InterestProfile.BulkUpsert → BatchRun.MarkSuccess → AuditLog.Create(best effort) → BatchLock.Release	行動ログから興味スコアを再計算する	不要
InterestBatch失敗	BatchRun.MarkFailed → AuditLog.Create(best effort) → BatchLock.Release	失敗理由を保存する	不要
Admin Bean作成	Bean.Create → AuditLog.Create(best effort)	Beanを登録する。RankTargetは作らない	不要
Admin Bean更新	Bean.Update → AuditLog.Create(best effort)	Beanを更新する	不要
Admin Bean公開	TxManager.WithinTx(Beans.UpdatePublished → RankTarget.FindOrCreate → RankTarget.UpdateActive) → AuditLog.Create(best effort)	Bean公開状態とランキング対象状態を一致させる	必須
Admin Bean非公開	TxManager.WithinTx(Beans.UpdatePublished → RankTarget.UpdateActive) → AuditLog.Create(best effort)	Bean非公開状態とランキング対象無効状態を一致させる	必須
Admin Article作成	Article.Create → AuditLog.Create(best effort)	Articleを登録する。RankTargetは作らない	不要
Admin Article更新	Article.Update → AuditLog.Create(best effort)	Articleを更新する	不要
Admin Article公開	TxManager.WithinTx(Article.UpdatePublished → RankTarget.FindOrCreate → RankTarget.UpdateActive) → AuditLog.Create(best effort)	Article公開状態とランキング対象状態を一致させる	必須
Admin Article非公開	TxManager.WithinTx(Article.UpdatePublished → RankTarget.UpdateActive) → AuditLog.Create(best effort)	Article非公開状態とランキング	必須

処理	Repository呼び出し順	何をしているか	transaction
		対象無効状態を一致させる	
Admin Relation作成	Bean.ExistsByID → Article.ExistsByID → BeanArticle.Create → AuditLog.Create(best effort)	BeanとArticleの関連を1件作成する	不要
Admin Relation削除	BeanArticle.Delete → AuditLog.Create(best effort)	BeanとArticleの関連を1件削除する	不要
Admin Relation差し替え	Bean.ExistsByID → 各Article.ExistsByID → TxManager.WithinTx(BeamArticle.ReplaceByBeanID) → AuditLog.Create(best effort)	指定Beanの関連Articleをまとめて差し替える	必須
Admin Relation並び替え	Bean.ExistsByID → 各Article.ExistsByID → TxManager.WithinTx(BeamArticle.ReplaceByBeanID) → AuditLog.Create(best effort)	関連Articleを指定順に差し替える	必須
Cleanup	RefreshToken.DeleteExpired → GuestSession.DeleteExpired → InterestProfile.DeleteExpired	期限切れデータを削除する	不要

4. エラー定義

UsecaseはHTTP statusを返さない。

Usecaseは下記のような業務エラーを返し、ControllerがHTTP statusへ変換する。

共通エラー

エラー名	何をしている時に起きるか	意味	Controllerでの変換目安
ErrInvalidInput	入力値を業務ルールで確認している時	入力値が不正	400
ErrUnauthorized	認証済みUserが必要な時	認証されていない	401
ErrForbidden	Admin権限が必要な時	権限がない	403
ErrNotFound	対象取得時	対象が存在しない	404
ErrRepositoryFailed	Repository呼び出し時	DB / Redisなどの永続化層エラー	500
ErrCreateFailed	作成処理時	DB作成に失敗	500
ErrUpdateFailed	更新処理時	DB更新に失敗	500
ErrDeleteFailed	削除処理時	DB削除に失敗	500
ErrConflict	重複作成など	既に存在している	409

Auth系エラー

エラー名	何をしている時に起きるか	意味	Controllerでの変換目安
ErrEmailAlreadyExists	Signup時	emailが既に使われている	409
ErrInvalidCredentials	Login時	emailまたはpasswordが不正	401
ErrUserSuspended	Login時	停止中ユーザー	403
ErrUserDeleted	Login時	退会済みユーザー	403
ErrPasswordHashFailed	Signup時	password hash化失敗	500
ErrInvalidToken	Refresh時	tokenが不正	401
ErrRefreshTokenExpired	Refresh時	RefreshToken期限切れ	401
ErrRefreshTokenRevoked	Refresh時	RefreshToken失効済み	401
ErrRefreshTokenReuseDetected	Refresh時	使用済みRefreshTokenが再利用された	401
ErrRevokeFailed	Logout / reuse検知時	token失効処理失敗	500

GuestSession系エラー

エラー名	何をしている時に起きるか	意味	Controllerでの変換目安
<code>ErrInvalidSession</code>	GuestSession取得時	session keyが不正	401
<code>ErrGuestSessionExpired</code>	GuestSession取得時	GuestSession期限切れ	401
<code>ErrGuestSessionNotFound</code>	GuestSession確認時	GuestSessionが存在しない	404

Content系エラー

エラー名	何をしている時に起きるか	意味	Controllerでの変換目安
<code>ErrBeanNotFound</code>	Bean取得時	Beanが存在しない、または非公開	404
<code>ErrArticleNotFound</code>	Article取得時	Articleが存在しない、または非公開	404
<code>ErrRankTargetNotFound</code>	保存・評価・イベント時	RankTargetが存在しない、または無効	404
<code>ErrInvalidContentType</code>	RankTarget / Ranking取得時	content_typeが不正	400
<code>ErrInvalidPagination</code>	一覧取得時	limit / offsetが不正	400
<code>ErrSlugAlreadyExists</code>	Article作成時	slugが既に存在する	409

Search / Event系エラー

エラー名	何をしている時に起きるか	意味	Controllerでの変換目安
<code>ErrInvalidSearchCondition</code>	検索時	検索条件が不正	400
<code>ErrHashFailed</code>	検索条件hash作成時	hash作成失敗	500
<code>ErrActorRequired</code>	Event記録時	UserIDまたはGuestSessionIDが必要	400
<code>ErrInvalidEventType</code>	Event記録時	event_typeが不正	400
<code>ErrStayTooShort</code>	stay記録時	滞在時間が3秒未満	400 または 204
<code>ErrDuplicateEvent</code>	dedup判定時	重複イベント	200 または 204
<code>ErrEventRecordFailed</code>	ActionEvent作成時	行動ログ記録失敗	500

Save / Rating系エラー

エラー名	何をしている時に起きるか	意味	Controllerでの変換目安
<code>ErrLoginRequired</code>	保存・評価時	Guestは保存・評価不可	401
<code>ErrSaveFailed</code>	保存時	保存処理失敗	500
<code>ErrSavedItemNotFound</code>	保存解除時	保存状態が存在しない	404
<code>ErrRemoveFailed</code>	保存解除時	保存解除失敗	500
<code>ErrInvalidRatingScore</code>	評価時	scoreが <code>1</code> / <code>-1</code> 以外	400
<code>ErrRatingFailed</code>	評価時	評価保存失敗	500

Ranking / Modal系エラー

エラー名	何をしている時に起きるか	意味	Controllerでの変換目安
<code>ErrRankingBuildFailed</code>	ランキング表示作成時	metricと実体の結合に失敗	500
<code>ErrInvalidModalTrigger</code>	Modal表示判定時	triggerが不正	400
<code>ErrModalBlocked</code>	Modal表示判定時	表示制限により出さない	200 または 204
<code>ErrNoModalCandidate</code>	Modal候補作成時	表示候補がない	204
<code>ErrModalDisplayNotFound</code>	Modal click / close時	表示ログが存在しない	404
<code>ErrRedisFailed</code>	Redis利用時	dedup / suppression / lock処理失敗	500

Batch / Admin / Audit系エラー

エラー名	何をしている時に起きるか	意味	Controllerでの変換目安
------	--------------	----	------------------

<code>ErrBatchAlreadyRunning</code>	Batch実行時	同じbatchが実行中	409
<code>ErrAggregateFailed</code>	Batch集計時	action_events集計失敗	500
<code>ErrScoreCalculateFailed</code>	Ranking / Interest計算時	score計算失敗	500
<code>ErrUpsertFailed</code>	metrics / interest保存時	upsert失敗	500
<code>ErrInvalidFilter</code>	Admin検索時	filter条件が不正	400
<code>ErrAuditLogFailed</code>	AdminAuditUsecaseで監査ログの一覧・詳細・検索を 主な処理として扱う時	audit log取得または検索に 失敗した	500

AuditLog.Create失敗時の扱いは、Login、Logout、Admin作成・更新・公開、Batch完了などの副作用としてAuditLog.Createを呼ぶ場合、AuditLog作成失敗では本体処理をrollbackしない。失敗はログ出力し、監視対象にする